Project Report: Fantasy LLM Classification

Team Leader: Shane Cumberbatch
Members: Nikolai Cupid, Kevaughna McClymont, Eric Kim

Problem Definition

To Curate our dataset, we picked ChatGPT, Perplexity, and Gemini as our LLMs. We asked each LLM 100 questions. The components used were LLM_name, LLM_input, LLM_output, and Category. To clean the data, Tokenization and lowercasing were done. Removing missing responses and duplicates were implemented,as well as, adding encoding to the LLM_name.

CNN/RNN Implementation and Training

We implemented CNN and RNN models through Python and PyTorch to find which of the 3 LLMs we picked was used for a prompt: ChatGPT, Gemini, and Perplexity. Our dataset was 300 responses to 100 fantasy football questions. Each fantasy football question was used in each LLM, generating us 300 responses. We cleaned that dataset, making sure there were no errors/duplicates. We then tokenized each response (LLM_output), and we split the dataset into 3 subsets. Training data which has 180 responses, validation which has 60 responses, and testing data that has another 60 responses totaling to original 300 in a 60/20/20 split. We also created a vocabulary from the training responses, converting every word into a numerical token ID. Every response was truncated or padded so it did not surpass a length of 256 tokens. We encoded the LLM names to GPT (0), Gemini (1), and Perplexity (2).

For our CNN model, we tried to find local patterns in text that could help differentiate between the different LLMs. We did this by converting token IDs into 64 dimensional word embedding. We also applied 32 filters with a kernel size of 3 to try and find patterns across neighboring words. We also added non-linerality so the model can learn more complex patterns. We also gave it the ability to pick up on the strongest feat from every response, a dropout rate to stop overfitting, and three output scores relating to ChatGPT, Gemini, and Perplexity

For our RNN model, we used a Long Short-Term Memory (LSTM) network. This helps us to process information sequentially and get potential relationships between words. Like the CNN, we converted token IDs into 64-dimensional embeddings, along with a dropout rate to help with overfitting and classified responses for each of the LLM categories. We used 64 hidden units to learn the sequential patterns in the LLMs responses along with handling responses that are padded.

As you can see the models were trained very similarly. The only key difference is that the CNN used 32 filters as opposed to the LSTMs 64 hidden units. Both models still used the Adam optimizer along with Cross-Entropy Loss to measure the classification errors.

RQ1: Can we identify which LLM generated a response?

```
CNN Test Accuracy: 0.8166666666666667

Classification Report:
              precision    recall  f1-score   support

         GPT       0.89      0.80      0.84        20
      Gemini       0.82      0.90      0.86        20
  Perplexity       0.75      0.75      0.75        20

    accuracy                           0.82        60
   macro avg       0.82      0.82      0.82        60
weighted avg       0.82      0.82      0.82        60


Confusion Matrix:
[[16  1  3]
 [ 0 18  2]
 [ 2  3 15]]
```

```
LSTM Test Accuracy: 0.6833333333333333

Classification Report:
              precision    recall  f1-score   support

         GPT       0.75      0.60      0.67        20
      Gemini       0.68      0.95      0.79        20
  Perplexity       0.62      0.50      0.56        20

    accuracy                           0.68        60
   macro avg       0.68      0.68      0.67        60
weighted avg       0.68      0.68      0.67        60

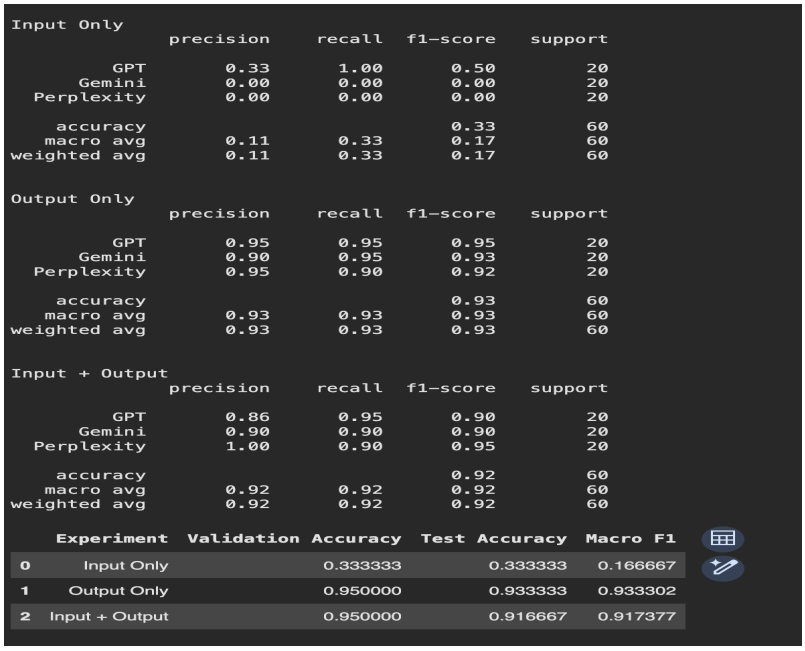

Confusion Matrix:
[[12  3  5]
 [ 0 19  1]
 [ 4  6 10]]
```

Yes you can identify which LLM generated a response. Although the accuracy is not the greatest, it is definitely accurate enough to say CNN and RNN can identify which LLM generated a response. CNN performed better with a 81.67% accuracy, while RNN gave a 68.33% accuracy. The CNN had a high recall rate for every LLM correctly identifying at least 75% of responses. While the RNN struggled having the lowest recall rate of about 50%, which is essentially a coinflip so that's not good. They follow similar trends in the f1-score, with the CNN finding a great balance between recall and precision while still remaining at high percentages. The RNN continues to struggle in that respect. Both models performed way better than just randomly picking though. Gemini was the easiest and Perplexity was by far the hardest for both models.

RQ2 — Does the user's prompt help identify the LLM?

Compare classifiers using: Input only; Output only; Input +Output Questions you may consider: i) Can the LLM be predicted from the user's prompt even before seeing its response? ii) Does including the input improve classification? iii) If input-only classification works surprisingly well, why might this happen?

Since we used the same prompts for each LLM, they ultimately did not help us identify the LLMs. For input only, we get an accuracy of 33%. This seems bad but it actually makes sense. Since all the LLMs take the same prompts, the model essentially has to guess the model based on the prompt. Since the model always defaults to picking ChatGPT (as it's the first LLM used for each prompt), the accuracy naturally comes out to 33%. Being as we use 3 LLMs total so ⅓ makes it 33%. That is also the reason why precision, recall, and f1-score are all 0. Taking strictly the output and input + output, the accuracy skyrockets to 95%.



Input Only

	precision	recall	f1-score	support
GPT	0.33	1.00	0.50	20
Gemini	0.00	0.00	0.00	20
Perplexity	0.00	0.00	0.00	20
accuracy			0.33	60
macro avg	0.11	0.33	0.17	60
weighted avg	0.11	0.33	0.17	60

Output Only

	precision	recall	f1-score	support
GPT	0.95	0.95	0.95	20
Gemini	0.90	0.95	0.93	20
Perplexity	0.95	0.90	0.92	20
accuracy			0.93	60
macro avg	0.93	0.93	0.93	60
weighted avg	0.93	0.93	0.93	60

Input + Output

	precision	recall	f1-score	support
GPT	0.86	0.95	0.90	20
Gemini	0.90	0.90	0.90	20
Perplexity	1.00	0.90	0.95	20
accuracy			0.92	60
macro avg	0.92	0.92	0.92	60
weighted avg	0.92	0.92	0.92	60

	Experiment	Validation Accuracy	Test Accuracy	Macro F1
0	Input Only	0.333333	0.333333	0.166667
1	Output Only	0.950000	0.933333	0.933302
2	Input + Output	0.950000	0.916667	0.917377

In-depth Analysis

When looking at the results for CNN and RNN, it shows that accuracy of identifying which LLM gave a response drastically improves from 33.3%(purely guessing). For both models, they found that Gemini was the easiest to identify and Perplexity was the hardest. Between the two models, they could recall Gemini 90-95% of the time, while Perplexity was only about 50-75%. This seemed interesting since Perplexity seemed the most distinct in its responses. This could be because many of the outputs were tables of text, which could be jumbled up when transferring over to the excel sheet. Something we also noticed about perplexity was that towards the last 10-15 input questions, it started using “This is my judgement” every couple of responses. This could possibly be a reason as to why the accuracy was lower. ChatGPT and Gemini gave more concise outputs that didn’t really deviate other than getting shorter responses after we kept asking it more and more questions.

In RQ2,when we combined the input and outputs, we expected the results to show a slight increase for accuracy, but there was actually a slight decrease. The reason for this was because our inputs for each LLM were the exact same, so the inputs do not really help identify which LLM before its response/output. When we added the input with the output, any result that the models were deciding between could have been mixed up because of that extra aspect. The difference was very small, but seemed important to point out. Had we experimented with trying to give similarly worded inputs, maybe the input + output results would have given an improvement.

Lessons&Experience

When it comes to trying to find what LLM generated a response through different means of local text pattern recognition, we found that the CNN performed better as opposed to the sequential

relationships garnered by the LSTM. We also learned the importance of having enough data to be both properly trained and validated. As well as how important the proportion is in training/validating/testing. Even a 5% shift can have large consequences, especially when you have a smaller dataset.